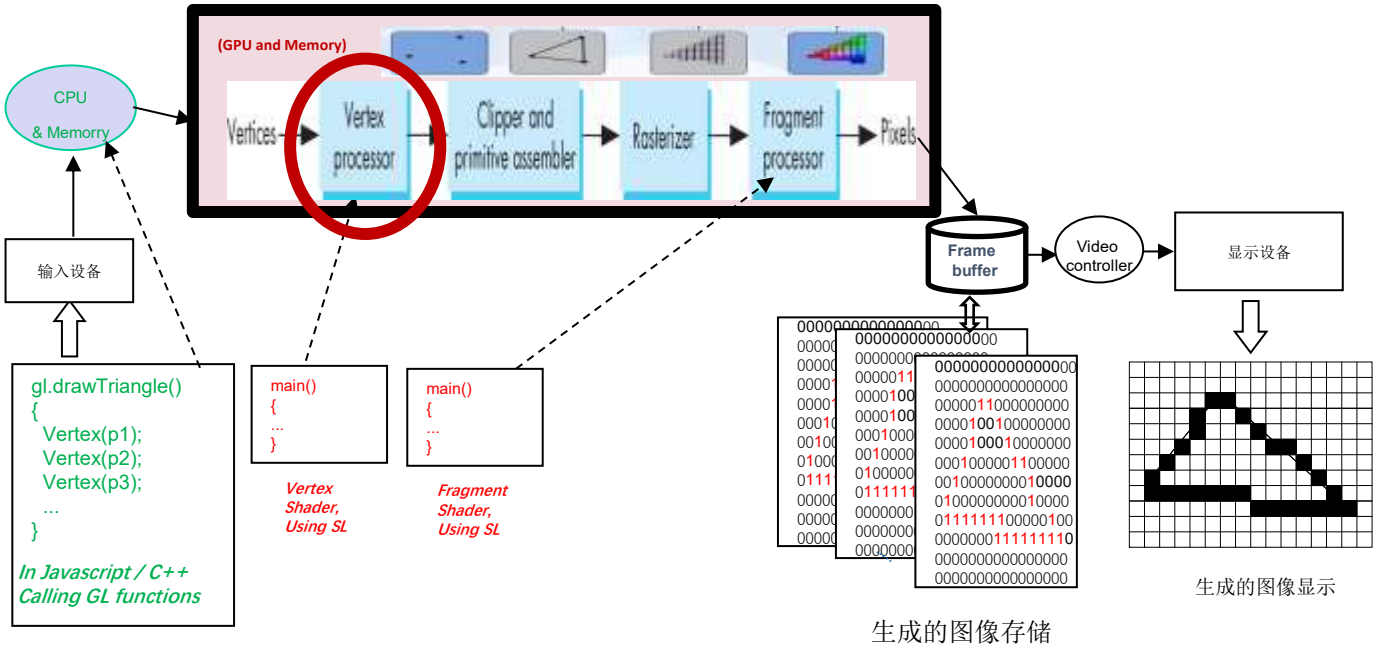
The University of New Mexico

Recap

- The Programmable Rendering Pipeline 可编程渲染管线的架构
  - Performance is achieved by using GPU rather than CPU,GPU does all rendering,
  - Programmer Control GPU through shaders
    - Application's job (run at CPU) is to send data to GPU
    - Vertext Shader run at per vertex, fragment shader run at per fragment concurrently



The University of New Mexico

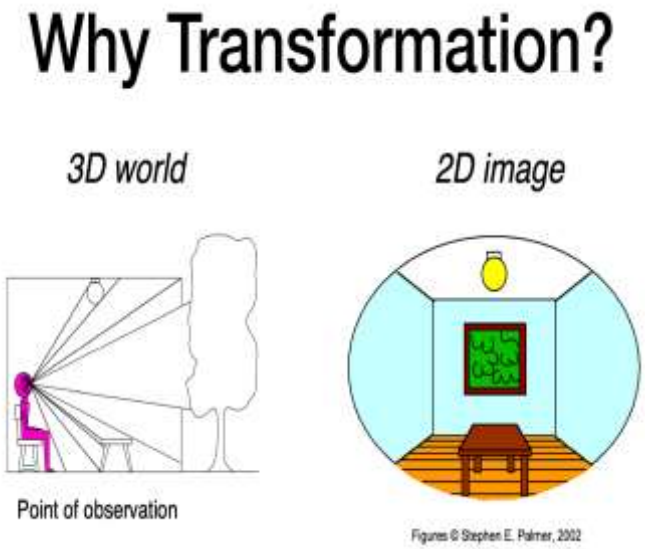
Today's Agenda

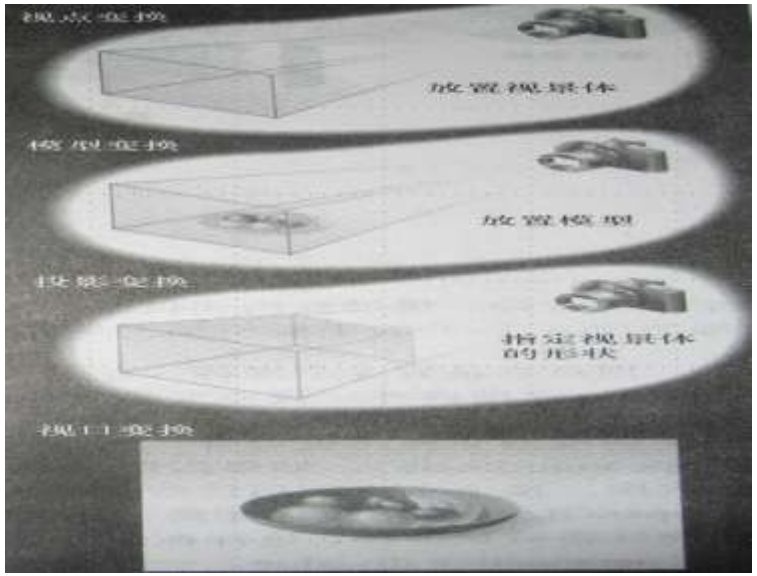
- 目标:

1.掌握几何图形(点和向量)的表示: 向量坐标, 标架, 齐次坐标

2.掌握五种"标准变换", 熟悉"串联变换(多个标准变换的顺序乘积)"

3.**应用**"串联变换"表达各种变换:非标变换, 渐增变换, 观察变换.....







Outline

- **Geometry Representation（表示）**
  - Vector space, Coordinate System, Change of Coordinate
  - Affine space, Frames System, Change of Frame
  - Homogeneous Coordinate*
- **Affine Transformations*（仿射变换）**
  - Five Standard Transformation（标准变换）
  - Concatenation Transformation*（串联变换）
- **Applying Concatenation Transformation（应用串联变换）**
  - Eg1:Non-standard transformation非标变换
  - Eg2:Cumulative transformation累积变换
  - ……

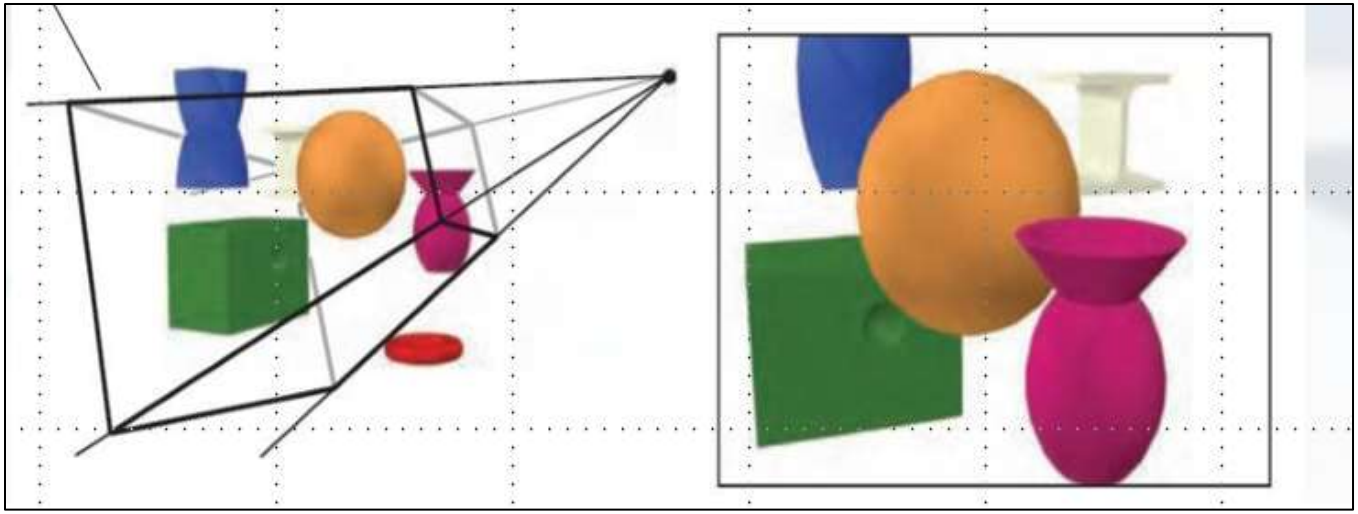
The University of New Mexico

Representation

- Need a frame of reference to relate points and objects to our physical world.（需要一个“参考框架”来将点和对象与我们的物理世界联系起来）

  For example, where is a point?

  - Can’t answer without **a reference system(参照系统)**





Outline

- **Representation（点/向量在物理空间的表示）**
  - Vector space, Coordinate System, Change of Coordinate
  - Affine space, Frames System, Change of Frame
  - Homogeneous Coordinate
- **Affine Transformations*（仿射变换）**
  - Five Standard Transformation（标准变换）
  - Concatenation Transformation*（串联变换）
- **Applying Transformations（变换的应用）**
  - Non-standard transformation非标变换
  - Cumulative transformation累积变换
  - **……**



Vector space, Coordinate System, Change of Coordinate

➢Coordinate Systems（坐标系统）

- Consider a basis $v_1, v_2, \ldots, v_n$, ,
- A vector is written $v=\alpha_1 v_1+ \alpha_2 v_2 +\ldots.+\alpha_n v_n$
- The list of scalars $\{\alpha_1, \alpha_2, \ldots. \alpha_n\}$is the representation of v with respect to the given basis,
- write the representation as a row or column array of scalars

$$\boldsymbol{a} =[\alpha_1 \;\; \alpha_2 \;\; \ldots. \alpha_n]^T = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ . \\ \alpha_n \\ \end{bmatrix}$$

Example: $v=2v_1+3v_2-4v_3$

$\mathbf{a}=[2 \;\; 3 \;\; -4]^T$

Note： that this representation is with respect to a particular basis

The University of New Mexico

Vector space, Coordinate System, Change of Coordinate(cont.)

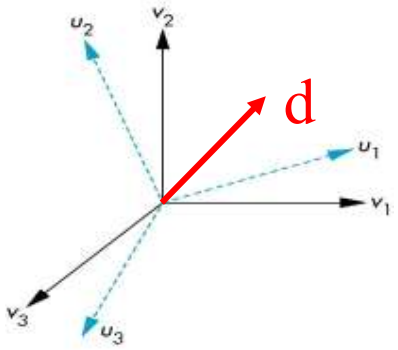
- **Change of Coordinate Systems(坐标系转换)**

  - Consider two representations of the same vector d with respect to two different bases.

The representations are:

$$\mathbf{a}=[\alpha_1\ \alpha_2\ \alpha_3]$$

$$\mathbf{b}=[\beta_1\ \beta_2\ \beta_3]$$



where

$$\mathbf{d}=\alpha_1 v_1+\alpha_2 v_2+\alpha_3 v_3=[\alpha_1\ \alpha_2\ \alpha_3]\,[v_1\ v_2\ v_3]^{T}$$

$$\mathbf{d}=\beta_1 u_1+\beta_2 u_2+\beta_3 u_3=[\beta_1\ \beta_2\ \beta_3]\,[u_1\ u_2\ u_3]^{T}$$

Vector space, Coordinate System, Change of Coordinate(cont.)

The University of New Mexico

- **Change of Coordinate Systems(坐标系统的转换)**
  - **Representing second basis in terms of first:**

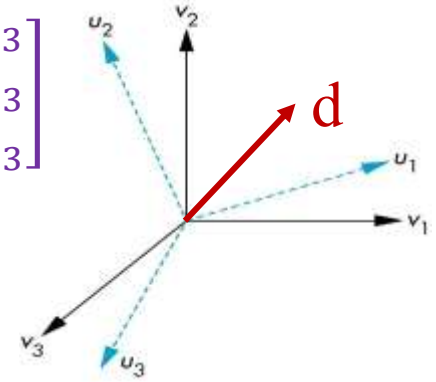
Each of the basis vectors, u1,u2, u3, are vectors that can be represented in terms of the first basis v1,v2,v3

➢ *已知:* $d=\alpha_1 v_1+\alpha_2 v_2+\alpha_3 v_3=[\alpha_1\ \alpha_2\ \alpha_3][v_1\ v_2\ v_3]^T$ $\mathbf{a}=[\alpha_1\ \alpha_2\ \alpha_3]$, $\mathbf{v}=[v_1\ v_2\ v_3]^T$

$d=\beta_1 u_1+\beta_2 u_2+\beta_3 u_3=[\beta_1\ \beta_2\ \beta_3][u_1\ u_2\ u_3]^T$ $\mathbf{b}=[\beta_1\ \beta_2\ \beta_3]$, $u=[u_1\ u_2\ u_3]^T$

Let:
$$u_1=\gamma_{11}v_1+\gamma_{12}v_2+\gamma_{13}v_3$$
$$u_2=\gamma_{21}v_1+\gamma_{22}v_2+\gamma_{23}v_3$$
$$u_3=\gamma_{31}v_1+\gamma_{32}v_2+\gamma_{33}v_3$$

$$\mathbf{M}=\begin{bmatrix}\gamma_{11} & \gamma_{12} & \gamma_{13}\\ \gamma_{21} & \gamma_{22} & \gamma_{23}\\ \gamma_{31} & \gamma_{32} & \gamma_{33}\end{bmatrix}$$



Proof: for $\mathbf{d}=\mathbf{bu}$, **and** $\mathbf{u}=\mathbf{Mv}$

so: $\mathbf{d}=\mathbf{b}(\mathbf{Mv})=\mathbf{bMv}$, and: $\mathbf{d}=\mathbf{av}$

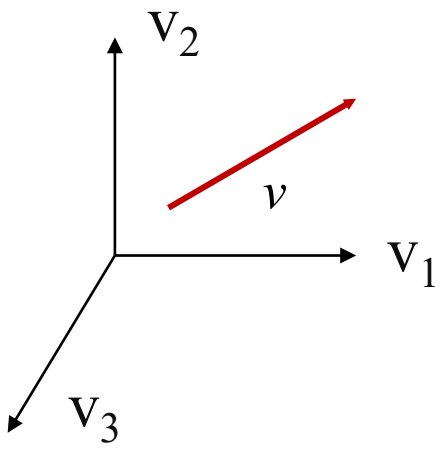
=> $\mathbf{a}=\mathbf{b}\,\mathbf{M}$ *// here a , b are row vector representation*

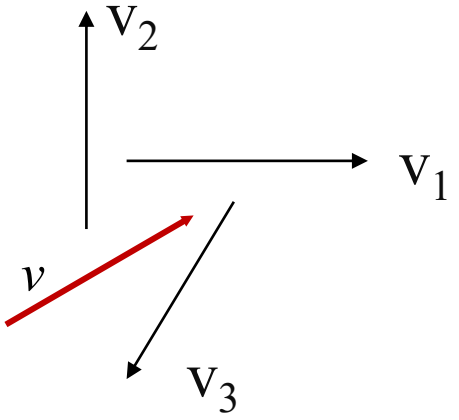
or: $\mathbf{b}=\mathbf{a}*\mathbf{M}^{-1}$

Vector space, Coordinate System, Change of Coordinate(cont.)

- Example: $v=2v_1+3v_2+4v_3$ **v向量可表示为 a=[2 3 4]^T**
  - The two case ,Which one is correct ?
    - **Both are**, because vectors have no fixed location

      (两种情况下，v向量具有相同的表示/系数，因为向量没有固定位置！)







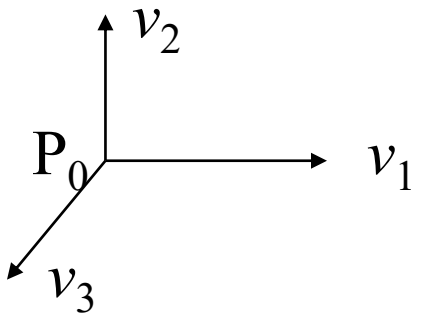
Outline

- **Representation（对象在物理空间的表示）**
  - Vector space, Coordinate System, Change of Coordinate
  - Affine space, Frames System, Change of Frame
  - Homogeneous Coordinate
- **Affine Transformations*（仿射变换）**
  - Five Standard Transformation（标准变换）
  - Concatenation Transformation*（串联变换）
- **Applying Concatenation Transformation（应用串联变换）**
  - Eg1:Non-standard transformation非标变换
  - Eg2:Cumulative transformation累积变换
  - Eg3:Instance transformation实例变换

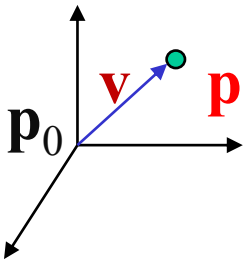
The University of New Mexico

Affine space, Frames System, Change of Frame

- Frames(标架）
  - If we work in an affine space（仿射空间）, can add a single point：the origin（原点）, to the basis vectors（基向量）, to form a frame（标架）
  - Frame（标架）determined by (P_0, v_1, v_2, v_3),



  - *Within this frame,*
    - Every **vector**（向量）*can be written as* $v=\alpha_1 v_1+ \alpha_2 v_2 +....+\alpha_n v_n$
    - Every **point**（点）*can be written as* $P = P_0 + \beta_1 v_1+ \beta_2 v_2 +....+\beta_n v_n$



The University of New Mexico

Affine space, Frames System, Change of Frame(cont.)

➢Use Frame（标架）Will Confusing Points and Vectors

Consider the **point** and the **vector**, They appear to have the similar representations（如果只采用向量的**系数**表示图形对象，存在问题：向量和点的 表示相似，难以区别）

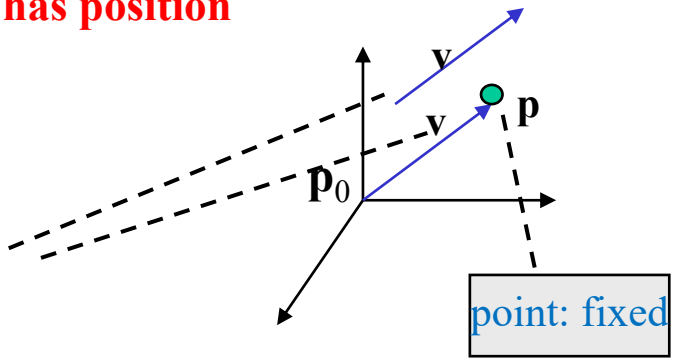
$v=\alpha_1 v_1+ \alpha_2 v_2 +\ldots.+\alpha_n v_n$ **v**: $[\alpha_1\ \alpha_2\ \ldots\ldots\alpha_n]$

$P = P_0+ \beta_1 v_1+ \beta_2 v_2 +\ldots.+\beta_n v_n$ **p**: $[\beta_1\ \beta_2\ \ldots\ldots\beta_n]$

what confuses the point with the vector?

➢**A vector has no position， but points has position**

Vector can be placed anywhere





Affine space, Frames System, Change of Frame(cont.)

➢Redefine Frame Representation(重定义标架表示)

- ➢对三维空间的表示增加一个维度，表示为四维坐标(齐次坐标)
  - ➢前三个维度表示系数，第4个维度用来区分对象：0表示“向量”，1表示“点”！

✓If we define $0 \bullet P = \mathbf{0}$ and $1 \bullet P = P$,Then

$$v = \alpha_1 v_1 + \alpha_2 v_2 + \alpha_3 v_3 + 0 \bullet P_0 = [\alpha_1\ \alpha_2\ \alpha_3\ 0\ ][v_1\ v_2\ v_3\ P_0]^T$$

$$P = \beta_1 v_1 + \beta_2 v_2 + \beta_3 v_3 + 1 \bullet P_0 = [\beta_1\ \beta_2\ \beta_3\ 1\ ][v_1\ v_2\ v_3\ P_0]^T$$

✓现在，向量和点表示如下：

$$\textbf{Vector:}\quad v = [\alpha_1\ \alpha_2\ \alpha_3\ 0\ ]^T$$

$$\textbf{Point:}\quad p = [\beta_1\ \beta_2\ \beta_3\ 1\ ]^T$$

Affine space, Frames System, Change of Frame(cont.)

The University of New Mexico

➢Change of Frames (标架间的转换)

- Representing One Frame in Terms of the Other Frame

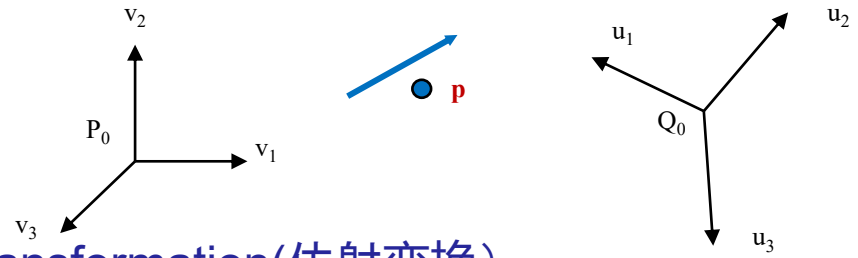
- Extending what we did with change of bases

Let: $u_1 = \gamma_{11}v_1+\gamma_{12}v_2+\gamma_{13}v_3 + 0\ P_0$

$u_2 = \gamma_{21}v_1+\gamma_{22}v_2+\gamma_{23}v_3 + 0\ P_0$

$u_3 = \gamma_{31}v_1+\gamma_{32}v_2+\gamma_{33}v_3 + 0\ P_0$

$Q_0 = \gamma_{41}v_1+\gamma_{42}v_2+\gamma_{43}v_3 + \gamma_{44}P_0$



- Defining a 4 x 4 matrix, specifies an affine transformation(仿射变换)

$$\mathbf{M} = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} & 0 \\ \gamma_{21} & \gamma_{22} & \gamma_{23} & 0 \\ \gamma_{31} & \gamma_{32} & \gamma_{33} & 0 \\ \gamma_{41} & \gamma_{42} & \gamma_{43} & 1 \end{bmatrix}$$

- Within the two frames, any **point** or **vector** has a representation of the same form, where $\alpha_4=1,\ \beta_4=1$ for points and $\alpha_4=1,\ \beta_4=0$ for vectors .

$a=[\alpha_1\ \alpha_2\ \alpha_3\ \alpha_4]$ in the first frame $(v_1, v_2, v_3, P_0)^T$

$b=[\beta_1\ \beta_2\ \beta_3\ \beta_4]$ in the second frame $(u_1,u_2,u_3,Q_0)^T$

➢**Proof**: for $\mathbf{P} = \mathbf{bu}$, **and** $\mathbf{u} = \mathbf{Mv}$ *//P is point or vector*

so: $\mathbf{P} = \mathbf{b(Mv)} = \mathbf{bMv}$, and: $\mathbf{P} = \mathbf{av}$ *//here a,b are row vector representation*

$\Rightarrow$ $a=b\,M$



Affine space, Frames System, Change of Frame(cont.)

➢Affine Transformations（仿射变换）

In Euclideean geometry ,an affine transformation is a geometric transformation That preserves lines and parallelism, but not ecessarily distances and angles.

- Every affine transformation preserves lines（仿射变换具有保线性(变换前是直线的，变换后依然是直线；变换前是平行线的，变换后依然是平行线),但未必保持距离和角度）
- Every linear transformation is equivalent to a change in frames（每个线性变换等价于一个框架变换M）
  - ✓ 例如：用“行向量”表示顶点，则变换表示为

$a=b\,M$ //变换矩阵M将齐次坐标 b转换为齐次坐标 a，这里a, b是1*4的行向量

$$\mathbf{M} = \begin{bmatrix} \gamma_{11} & \gamma_{12} & \gamma_{13} & 0 \\ \gamma_{21} & \gamma_{22} & \gamma_{23} & 0 \\ \gamma_{31} & \gamma_{32} & \gamma_{33} & 0 \\ \gamma_{41} & \gamma_{42} & \gamma_{43} & 1 \end{bmatrix}$$

- However, an affine transformation has only 12 degrees of freedom because 4 of the elements in the matrix are fixed , and are a subset of all possible 4 x 4 linear transformations（每个仿射变换，对应于一个4*4矩阵，但仅有12个自由度）

The University of New Mexico

Outline

- **Representation（对象在物理空间的表示）**
  - Vector space, Coordinate System, Change of Coordinate
  - Affine space, Frames System, Change of Frame
  - **Homogeneous Coordinate,**
- **Affine Transformations*（仿射变换）**
  - Five Standard Transformation（标准变换）
  - Concatenation Transformation*（串联变换）
- **Applying Concatenation Transformation（应用串联变换）**
  - Eg1:Non-standard transformation非标变换
  - Eg2:Cumulative transformation累积变换
  - ……



Homogeneous Coordinate

- **Homogeneous coordinates(齐次坐标)**
  - **齐次坐标是一种数学工具，用于投影几何中，通过将一个原本是n维的向量表示为一个n+1维向量，以简化几何变换和操作.**
  - 齐次坐标的概念由奥古斯特·费迪南德·莫比乌斯在1827年引入，并在多个领域中得到广泛应用，包括电脑图形学和3D电脑视觉。

- **齐次坐标 有四个元素 (X_h, Y_h, Z_h, w) ，w被称为齐次项，**
  - w=0： 表示”向量vector”
  - w!=0 ： 表示“点 point”

The University of New Mexico

womogeneous Coordinate

- **Homogeneous coordinates(齐次坐标)**
  - **点的齐次坐标的表示不是唯一的！**

    Ex. 二维点P(1，2)的齐次坐标可表示为

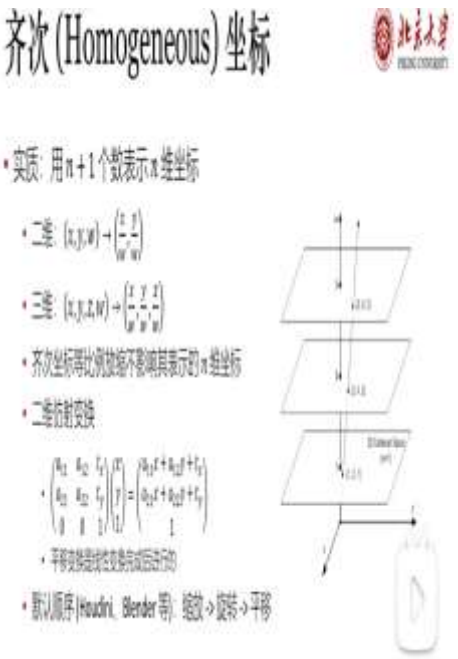
    (1，2，1) ，

    (2，4，2) ，

    (3，6，3), ...

  - **笛卡尔坐标（x,y,z）和齐次坐标(Xw Yw Zw w))的转换**
    - **X= X_w/w,**
    - **Y=Y_w/w,**
    - **Z= Z_w/w**

  - **“规格化齐次坐标”**
    - 当w=1时，点的齐次坐标被称为规格化齐次坐标
- //图形顶点一般都采用规格化齐次坐标来表示为（x,y,z,1）





The University of New Mexico

Homogeneous Coordinate(cont.)

- **Why Homogeneous coordinates ?**
  - **Homogeneous coordinates are key to all computer graphics systems:**
    - Hardware pipeline works with 4 dimensional representations .（用四维表示三维空间对象）
    - All standard transformations (rotation, translation, scaling) can be implemented with matrix multiplications using 4 x 4 matrices（用4 x 4矩阵的乘实现各种标准变换）



Outline

- **Representation*（对象在物理空间的表示）**
  - Vector space, Coordinate System, Change of Coordinate
  - Affine space, Frames System, Change of Frame
  - Homogeneous Coordinate*
- **Affine Transformations*（仿射变换）**
  - Five Standard Transformation（五种标准变换）
  - Concatenation Transformation*（串联变换）
- **Applying Concatenation Transformation（应用串联变换）**
  - Eg1:Non-standard transformation非标变换
  - Eg2:Cumulative transformation累积变换
  - ……

The University of New Mexico

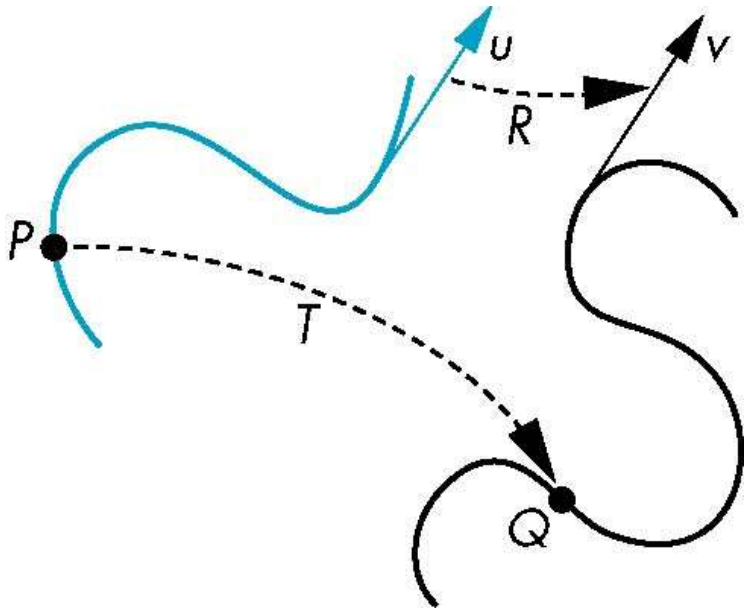
Transformations*（变换）

➢What is Transformation?

➢*maps points to other points* or *maps vectors to other vectors*

例如：把点P的表示，通过“T变换”，变为点Q的表示，

把向量u的表示，通过“R变换”，变为 向量v的表示，



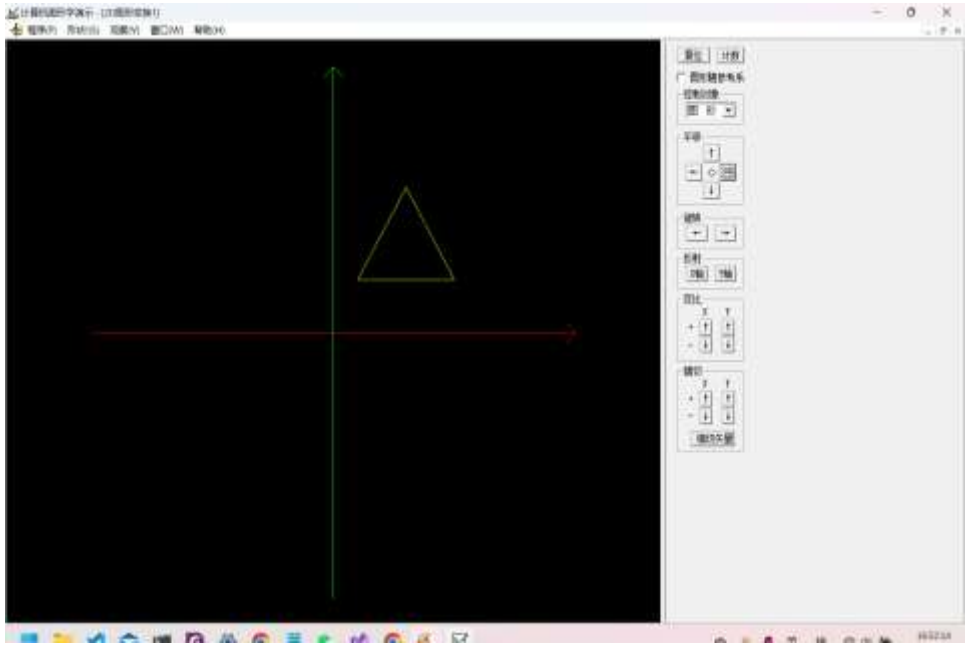
v=R(u)

Q=T(P)

The University of New Mexico

Standard Transformations*

- Introduce standard transformations（标准变换）and
- Derive homogeneous coordinate transformation matrices
  - **Rotation**
  - **Translation**
  - **Scaling**
  - **Reflection**
  - **Shear**



演示

（标准变换：以原点和坐标轴为参考点或参考轴的变换）

The University of New Mexico

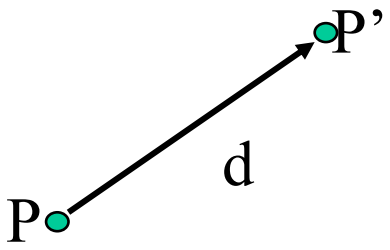
Standard Transformations*
1）Translation平移

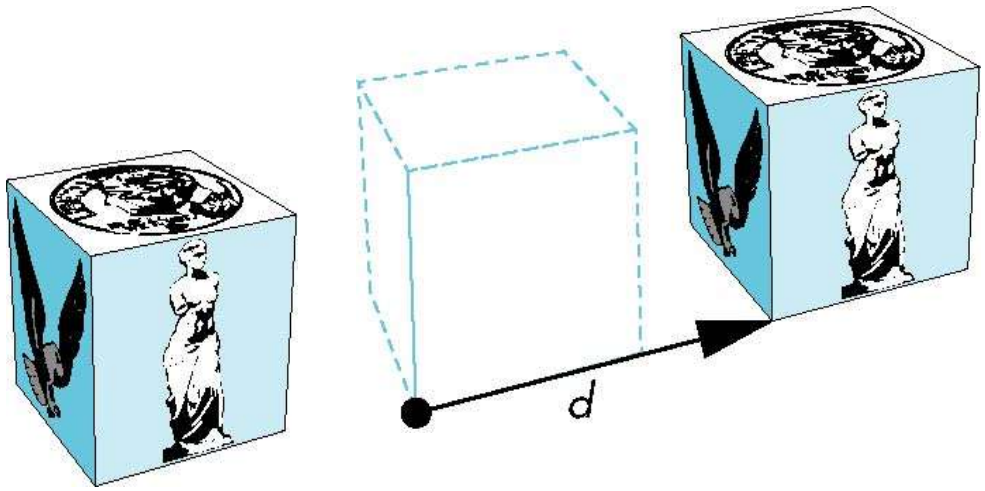
- Move (translate, displace) a point to a new location
- Displacement(移动）determined by a vector d
  But when we move many points，there is usually only one way
- Translation: every point displaced by same vector P'=P+d

$x'=x+d_x$

$y'=y+d_y$

$z'=z+d_z$





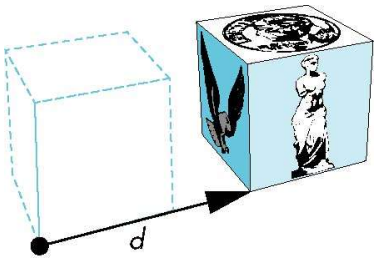
The University of New Mexico

Standard Transformations* 1）Translation平移（cont.）

➢Using the homogeneous coordinate representation in frame,

$$
\begin{aligned}
x' &= 1*x + 0*y + 0*z + d_x *1 \\
y' &= 0*x + 1*y + 0*z + d_y *1 \\
z' &= 0*x + 0*y + 1*z + d_z *1 \\
1 &= 0*x + 0*y + 0*z + 1 *1
\end{aligned}
$$





- express translation using a 4 x 4 matrix **T** in homogeneous coordinates

$$\mathbf{P'=T\,P} \quad \text{where} \quad \mathbf{T} = \mathbf{T}(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

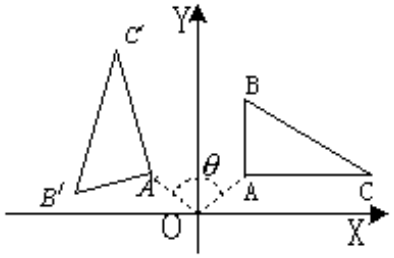
➢注意：P',P,d 都是列向量表示，T矩阵对P进行“左乘”，T是放射变换。

➢ This form in homogeneous coordinates is better for implementation!

Standard Transformations*
2）Rotation旋转

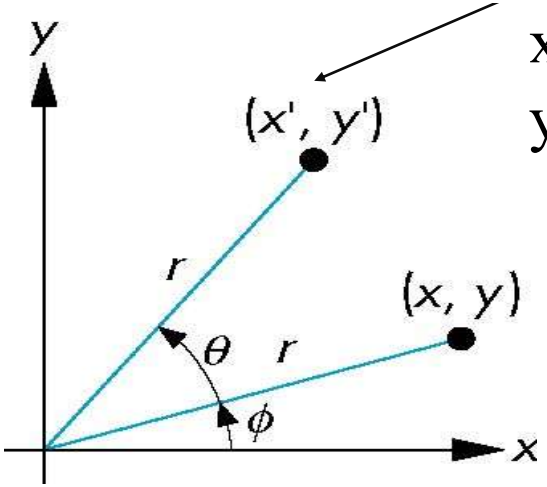
The University of New Mexico

- 二维平面上，绕原点的旋转变换
  - Consider rotation about the origin by θ degrees, radius stays the same, angle increases by θ
  - ***注意：逆时针方向旋转时，角度为正；顺时针方向旋转时，角度为负。***



相对原点旋转 θ角

$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$



$$x' = r\cos(\phi+\theta) = r\cos\phi\cos\theta - r\sin\phi\sin\theta$$
$$y' = r\sin(\phi+\theta) = r\cos\phi\sin\theta + r\sin\phi\cos\theta$$

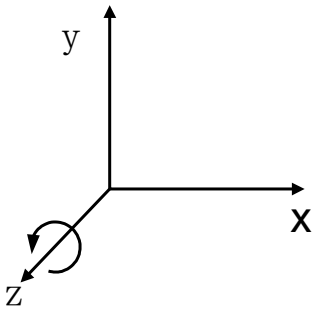
$$x = r\cos\phi$$
$$y = r\sin\phi$$

The University of New Mexico

Standard Transformations*
2）Rotation（cont.）

- 三维旋转，绕Z轴旋转
  - Rotation about z axis in three dimensions leaves all points with the same z, equivalent to rotation in two dimensions in planes of constant z

$$
\begin{aligned}
x' &= x*\cos\theta - y*\sin\theta + z*0 + 1*0 \\
y' &= x*\sin\theta + y*\cos\theta + z*0 + 1*0 \\
z' &= x*0 + y*0 + z*1 + 1*0 \\
1 &= x*0 + y*0 + z*0 + 1*1
\end{aligned}
$$



- in homogeneous coordinates

$$\mathbf{P'} = \mathbf{R_z}(\theta)\mathbf{P},$$

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The University of New Mexico

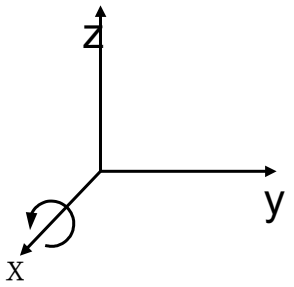
Standard Transformations*
2）Rotation(cont.)

- Rotation about x and y axes

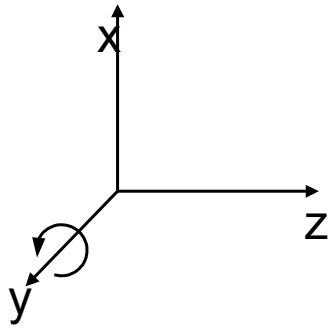
Same argument as for rotation about *z* axis

- For rotation about x axis, x is unchanged
- For rotation about y axis, y is unchanged

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

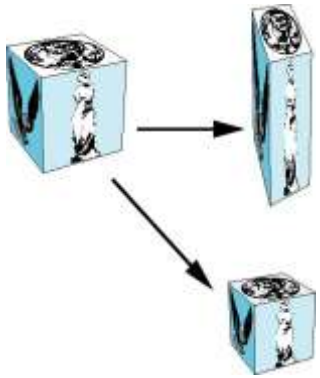


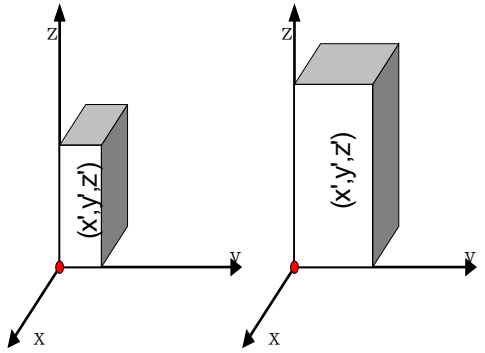
The University of New Mexico

Standard Transformations*
3) Scaling缩放/变比

- Expand or contract along each coordinate axis and fixed point of origin

$$x' = s_x * x$$
$$y' = s_y * y$$
$$z' = s_z * z$$
$$1 = 1*1$$





- in homogeneous coordinates
  - **P'=S P**

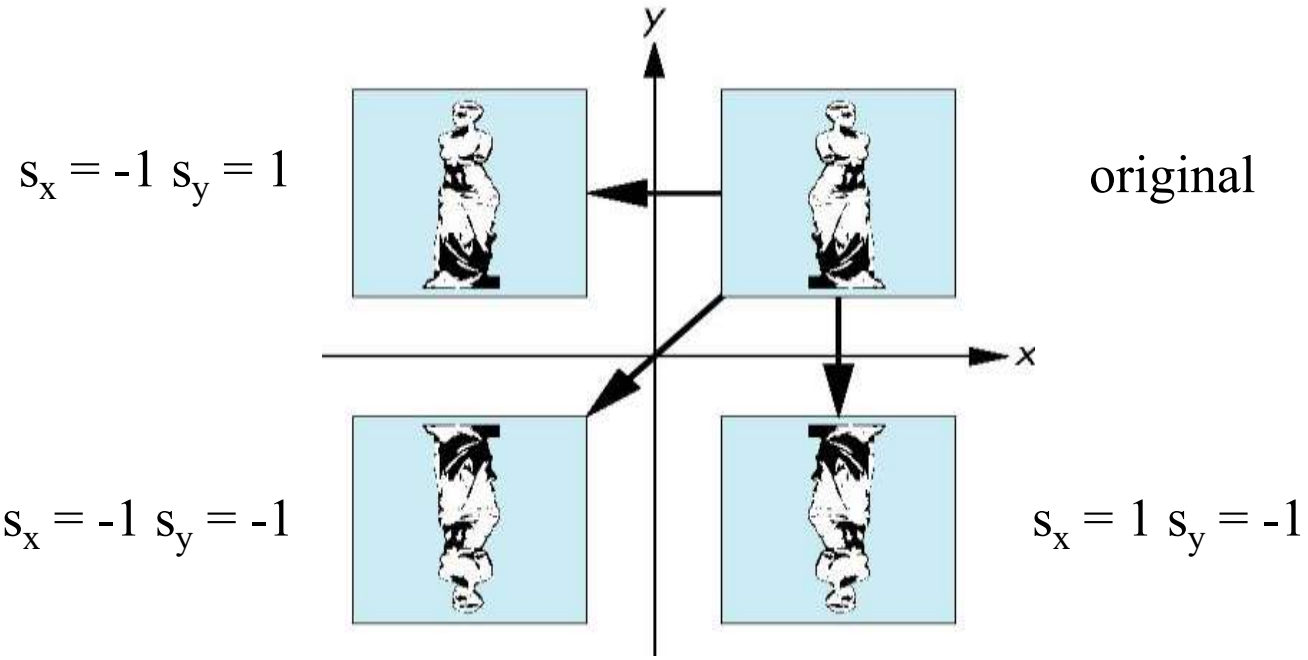
$$S = S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 注意:标准缩放是参考“原点”，沿着“X，Y，Z轴”进行缩放的！

Standard Transformations*
4）Reflection反射/对称

- 反射：参照坐标轴或原点进行
  - corresponds to **negative scale factors（缩放矩阵的某些因子为负来表示反射变换）**

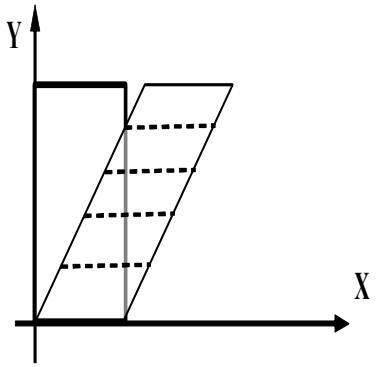
$$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



The University of New Mexico

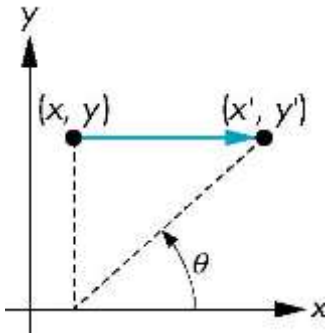
Standard Transformations*
5)Shear-错切

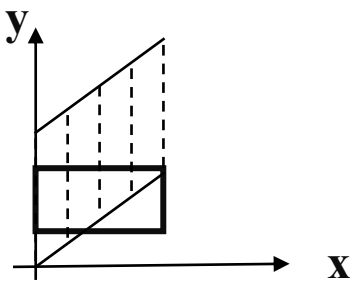
- 二维空间中，用齐次坐标表示的错切
  - P' =Shear(sh_x , sh_y)P $\begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$



x依y轴的错切：

- $x' = x + sh_x \cdot y$
- $y' = y$
- $1 = 1$





*Y*依X轴的错切

- $x_1 = x$
- $y_1 = sh_y \cdot x + y$
- $1 = 1$



Standard Transformations*
5)Shear-错切(cont.)

- 三维空间中，用齐次坐标表示的错切P' =Shear(sh_x , sh_y)P
  - 依赖轴:坐标保持不变的轴
  - 方向轴:坐标“参照依赖轴”呈线性变化的轴

X为依赖轴：

$$x' = x$$
$$y' = dx + y$$
$$z' = gx + z$$

Y为依赖轴：

$$x' = x + by$$
$$y' = y$$
$$z' = hy + z$$

z为依赖轴：

$$x' = x + cz$$
$$y' = y + fz$$
$$z' = z$$

$$T_{SHx} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ d & 1 & 0 & 0 \\ g & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{SHy} = \begin{bmatrix} 1 & b & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & h & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{SHz} = \begin{bmatrix} 1 & 0 & c & 0 \\ 0 & 1 & f & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Standard Transformations*

➢总结用齐次坐标表示变换形式如下：

- 用齐次坐标且”行向量”表示： **P’= P *M** //“右乘”变换矩阵M

$$\mathbf{p'}=[x'\ y'\ z'\ 1] \quad \mathbf{p}=[\,x\ y\ z\ 1\,] \qquad M_{4*4}=\begin{bmatrix} sx & d & g & 0 \\ b & sy & h & 0 \\ c & f & sz & 0 \\ tx & ty & tz & 1 \end{bmatrix}$$

- 用齐次坐标且”列向量”表示： **P’=M* P** //“左乘”变换矩阵M

$$\mathbf{p'}=\begin{bmatrix} x' \\ \boldsymbol{y'} \\ \boldsymbol{z'} \\ \mathbf{1} \end{bmatrix} \qquad M_{4*4}=\begin{bmatrix} sx & b & c & tx \\ d & sy & f & ty \\ g & h & sz & tz \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad \mathbf{p}=\begin{bmatrix} x \\ \boldsymbol{y} \\ \boldsymbol{z} \\ \mathbf{1} \end{bmatrix}$$



Outline

- **Representation（对象在物理空间的表示）**
  - Vector space, Coordinate System, Change of Coordinate
  - Affine space, Frames System, Change of Frame
  - Homogeneous Coordinate
- **Affine Transformations*（仿射变换）**
  - Five Standard Transformation（标准变换）
  - Concatenation Transformation*（串联变换）
- **Applying Concatenation Transformation（应用串联变换）**
  - Eg1:Non-standard transformation非标变换
  - Eg2:Cumulative transformation累积变换
  - ……



Concatenation Transformation

- Concatenation Transformation串联变换： 是多个标准变换的串联形式, 变换矩阵的累乘！
  - 采用齐次坐标表示，可将平移变换表示为矩阵乘，统一了变换的表示
  - 任意变换可表示为多个标准变换累乘(串联换矩阵M),作用于初始数据P0

✓好处：无需计算和保存中间过程产生的顶点数据Pi，可以减少数据量极大的顶点数据的存储和传输，从而大大提高顶点变换的效率！

$$\begin{cases} P_2 = M_1^1 \cdot P_1 + M_2^1 \\ P_3 = M_1^2 \cdot P_2 + M_2^2 \\ \cdots\cdots\cdots\cdots\cdots \\ P_n = M_1^n \cdot P_{n-1} + M_2^n \end{cases}$$

采用齐次坐标

$$\begin{cases} P_2 = M_1 P_1 \\ P_3 = M_2 P_2 \\ \cdots\cdots\cdots\cdots \\ P_n = M_n P_{n-1} \end{cases}$$

串联

$$P_n = M_{n-1} * \ldots * M_2 * M_1 * P_0 = M * P_0$$



Concatenation Transformation(cont.)

- 串联变换中矩阵乘的顺序Concatenation Order
  - many references use column matrices to represent points, note that "matrix on the right is the first applied"
    （采用列矩阵表示点，注意最右边矩阵是第1个应用的矩阵）

- Mathematically, the following are equivalent

$\mathbf{p'} = \mathbf{ABCp} = \mathbf{A(B(Cp))}$ //当列向量表示时，累乘顺序是左乘

$\mathbf{p'}^{T} = \mathbf{p}^{T}\mathbf{C}^{T}\mathbf{B}^{T}\mathbf{A}^{T}$ //行向量表示时：累乘顺序是右乘（万琳课件用之）

The University of New Mexico

Concatenation Transformation (cont.)

- The difficult part is *"how to **form** M"*

a desired Concatenation transformation is from the specifications in the application"

$$P_n = M * P_0$$

$$M = M_n * \ldots * M_2 * M_1$$



Outline

- **Representation（对象在物理空间的表示）**
  - Vector space, Coordinate System, Change of Coordinate
  - Affine space, Frames System, Change of Frame
  - Homogeneous Coordinate
- **Transformations*（变换）**
  - Five Standard Transformation（标准变换）
  - Concatenation Transformation*（串联变换）
- **Applying Concatenation Transformation（应用串联变换）**
  - Eg1:Non-standard transformation非标变换
  - Eg2:Cumulative transformation累积变换
  - .....

Non-standard transformation

- 补充：Inverses Transformation 逆变换

Although we could compute inverse matrices by general formulas, we can use simple geometric observations

- Translation: $\mathbf{T}^{-1}(d_x, d_y, d_z) = \mathbf{T}(-d_x, -d_y, -d_z)$
- Rotation: $\mathbf{R}^{-1}(\theta) = \mathbf{R}(-\theta)$, $\mathbf{R}^{-1}(\theta) = \mathbf{R}^{T}(\theta)$
  - Holds for any rotation matrix
  - Note that since $\cos(-\theta) = \cos(\theta)$ and $\sin(-\theta) = -\sin(\theta)$
- Scaling: $\mathbf{S}^{-1}(s_x, s_y, s_z) = \mathbf{S}(1/s_x, 1/s_y, 1/s_z)$

Non-standard transformation（cont.）

The University of New Mexico

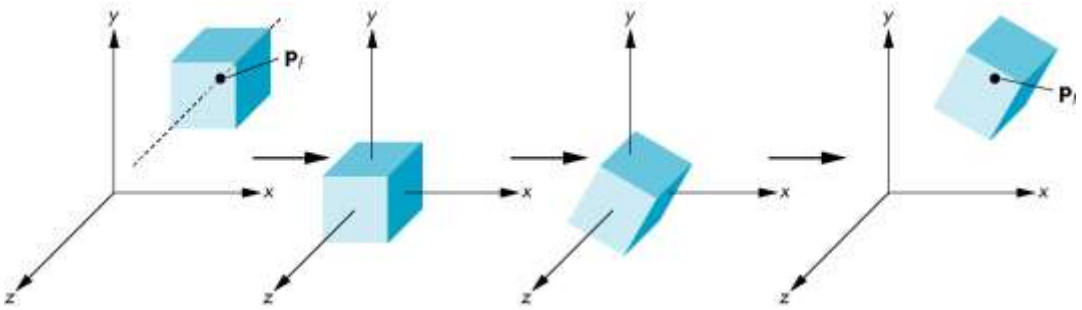
ex1: 绕任意点（非原点），且Z轴方向的旋转

Rotation About a Fixed Point other than the Origin

➢该变换表示为多个标准变换的串联变换矩阵:

$\mathbf{M} = \mathbf{T}(p_f)\ \mathbf{Rz}(\theta)\ \mathbf{T}(-p_f)$ //列向量表示:左乘顺序

1. Move fixed point to origin $\mathbf{T}(-p_f)$
2. Rotate θ $\mathbf{Rz}(\theta)$
3. Move fixed point back $\mathbf{T}(p_f)$



Non-standard transformation（cont.）

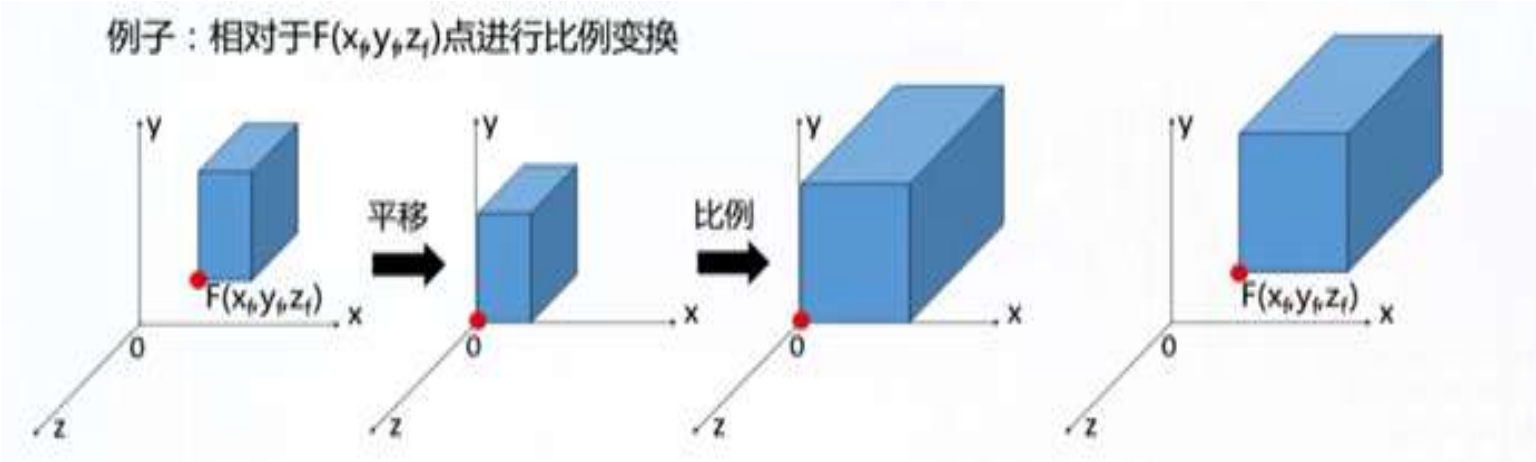
The University of New Mexico

Ex2: 依任意点的缩放(Scale About a Fixed Point other than the Origin)

➢该变换表示为多个标准变换矩阵累乘的串联矩阵:

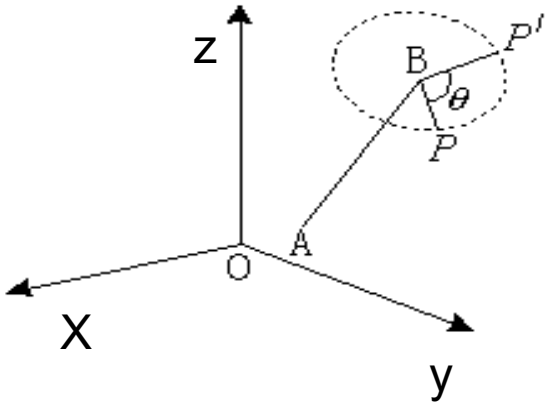
$\mathbf{M} = \mathbf{T}(X_f, Y_f, Z_f)\ \mathbf{S}(Sx,Sy,Sz)\ \mathbf{T}(-X_f, -Y_f, -Z_f)$ //列向量表示:左乘

1. Move fixed point to origin $\mathbf{T}(-X_f, -Y_f, -Z_f)$
2. Scale $\mathbf{S}(Sx,Sy,Sz)$
3. Move fixed point back $\mathbf{T}(X_f, Y_f, Z_f)$



Non-standard transformation（cont.)

- **Ex3:绕任意轴的旋转Rotation Transformation Around Any Axis**
- 如空间一点P(x_p, y_p, z_p)绕AB轴旋转角到P/(x_p', y_p', z_p')， 求Rab



$$\begin{bmatrix} x_p' \\ y_p' \\ z_p' \\ 1 \end{bmatrix} = R_{ab}(\theta) \begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix}$$

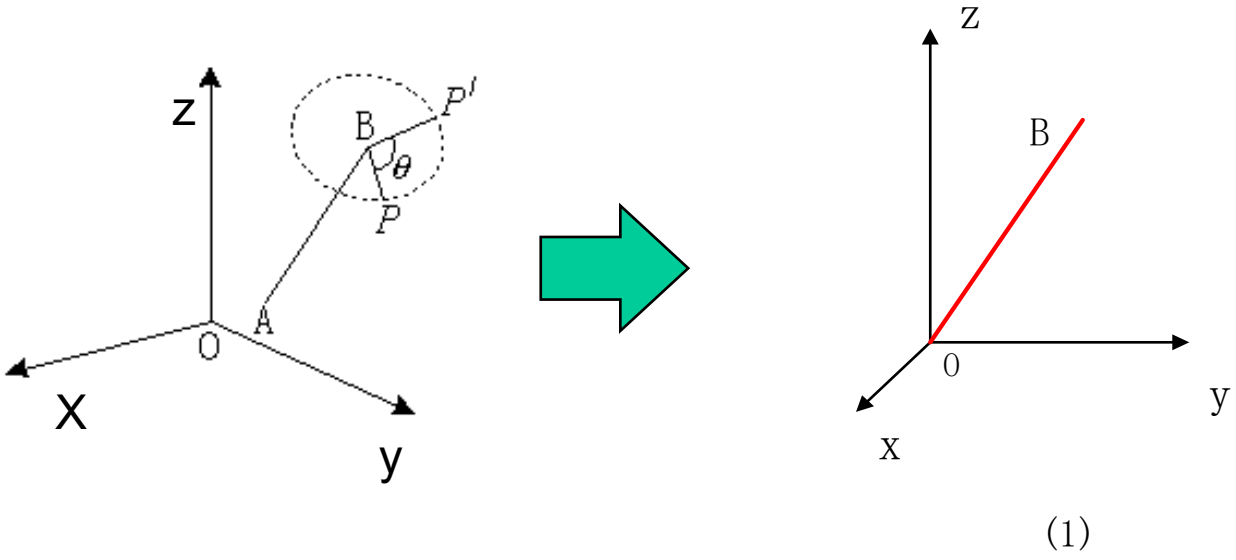
Non-standard transformation（cont.）

- **Ex: Rotation Transformation Around Any Axis（cont.）**

➢**Step1: 平移A点到原点**

将AB平移,使A点与坐标原点重合，得到OB

OB的方向数设为(a,b,c), $a^2+b^2+c^2=|OB|^2$



(1)

Non-standard transformation（cont.）

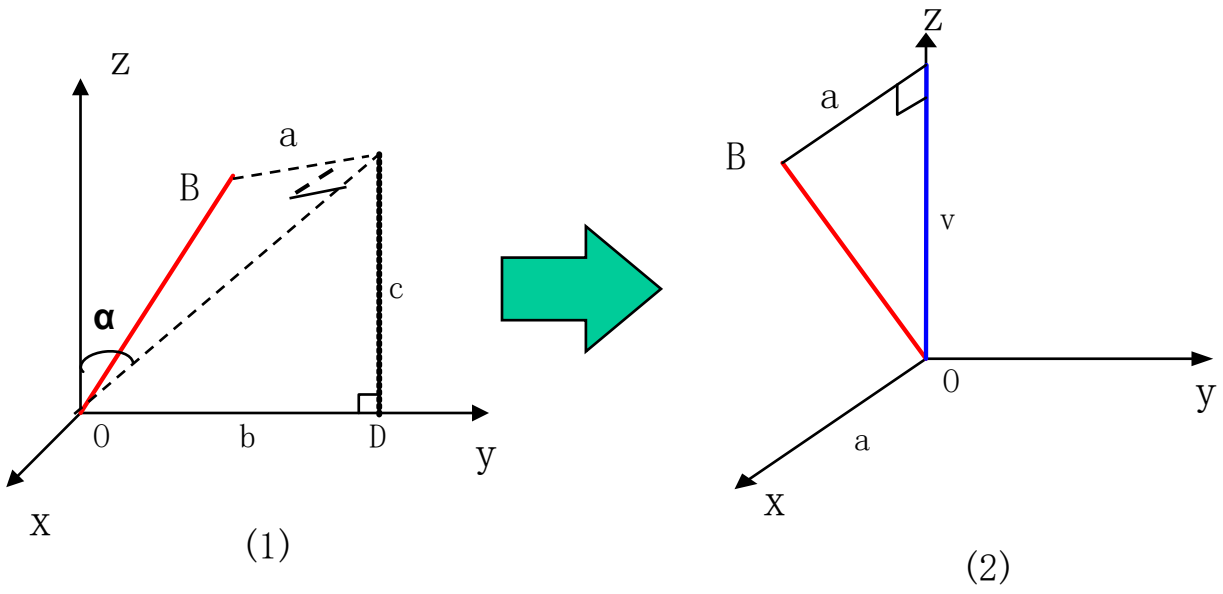
- **Ex: Rotation Transformation Around Any Axis（cont.）**

➢Step2: OB经绕X轴逆时针旋转到xoz面上

➢ **将OB绕x轴逆时针旋转α角，则OB旋转到XOZ平面上**

➢ **旋转角α：将OB投影到YOZ面后的投影线和Z轴的夹角，也是OB旋转的角度。**

➢ **v值：$v^2=b^2+c^2$**



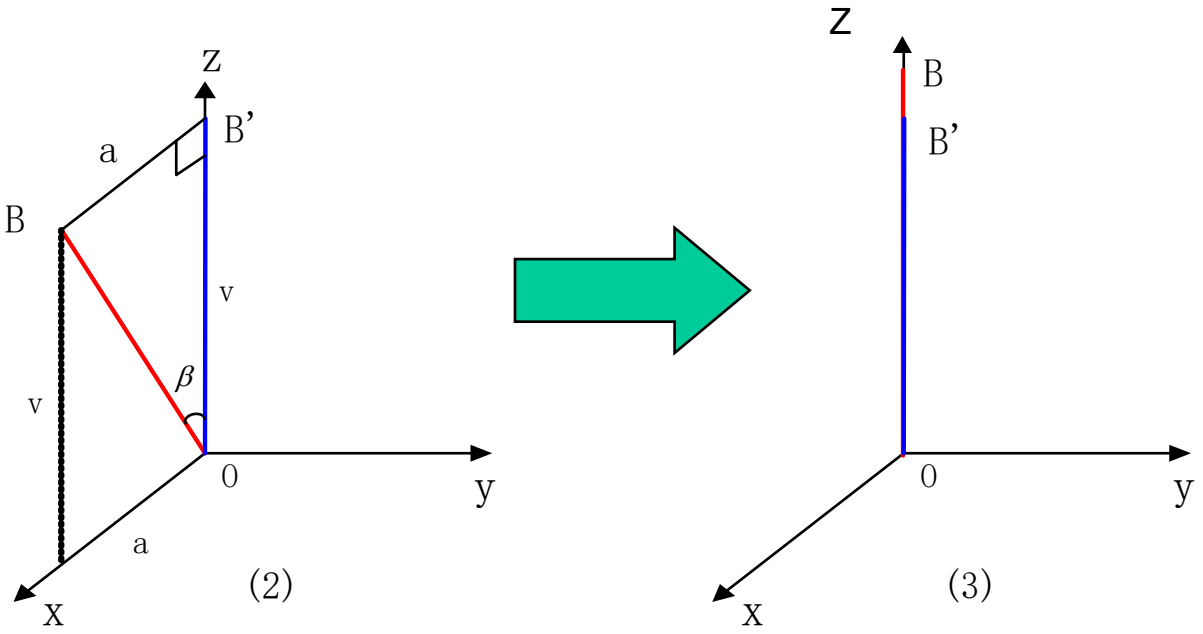
Non-standard transformation（cont.）

The University of New Mexico

- **Ex: Rotation Transformation Around Any Axis（cont.）**

➢Step3：**将OB绕Y轴顺时针旋转到Z轴上**

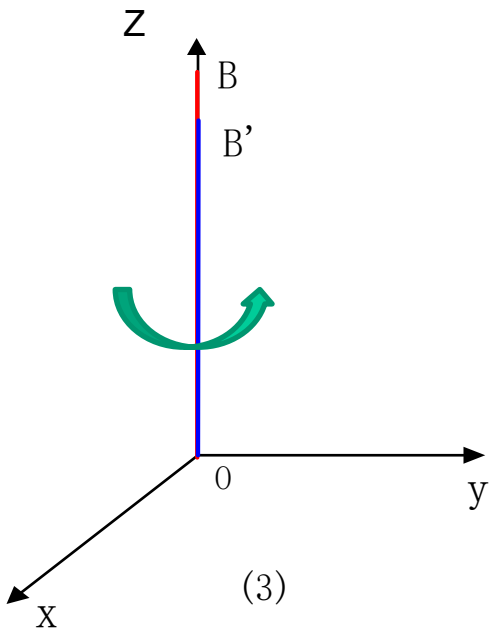
- ➢**将OB绕Y轴顺时针旋转β角，则OB旋转到Z轴上**
- ➢**旋转角β：将OB绕Y轴顺时针旋转的角度**
- ➢**数量关系：$v^2=b^2+c^2$ ， $OB^2= a^2+v^2 =a^2 + b^2+c^2$**



Non-standard transformation（cont.）

- **Ex: Rotation Transformation Around Any Axis（cont.）**

➢Step4：物体绕OB轴（Z轴）作标准旋转

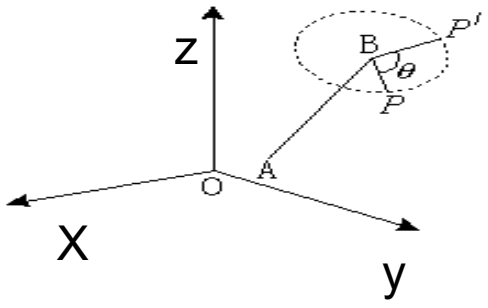


Non-standard transformation（cont.）

The University of New Mexico

- **Ex: Rotation Transformation Around Any Axis（cont.）**
  - Step5~Step7：逆变换
    - **作前面step3, step2,step1对应的逆变换**
    - 按顺序从右边到左排列这七个标准变换矩阵相乘，可求出Rab

$$T_{Rab} = T_{tA}^{-1} T_{Rx}^{-1} T_{Ry}^{-1} T_{Rz} T_{Ry} T_{Rx} T_{tA}$$



$$\begin{bmatrix} x_p' \\ y_p' \\ z_p' \\ 1 \end{bmatrix} = R_{ab}(\theta) \begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix}$$



Non-standard transformation（cont.）

➢Practice

对2D空间中的顶点，请写出以下对称变换所对应的串联矩阵

- 点 (2,3)对称
- 轴 y=2对称
- 轴 y=2x对称
- 轴y=ax+b对称

- **using Homogeneous Coordinate and column vector**

注意：采用齐次坐标表示和列向量时，矩阵乘顺序是“左乘”

提示: Non-Standard Transformation Solutions

1. *首先，T1-把”非标准”变换条件转换为标准变换条件*
2. *然后， T2-实施标准变换*
3. *最后，T3-逆变换回非标准条件下*

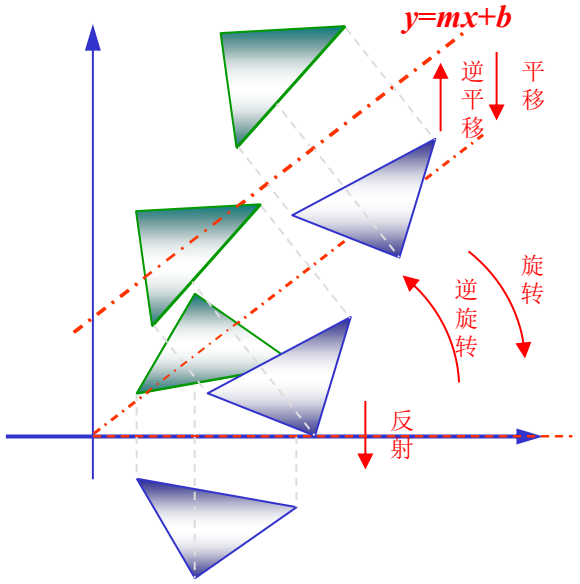
Non-standard transformation（cont.)

The University of New Mexico

➢ Practice(cont.)

推导关于任意轴y=ax+b的对称变换矩阵思路：

- ①平移反射轴使其经过原点；
- ②将反射轴旋转到坐标轴之一上，且进行关于坐标轴反射；
- ③利用逆旋转和平移变换将线置回原处。



- 串联矩阵M为：

M= T(0,b) •R(θ) • S(1,-1)• R(-θ) • T(0,-b)

1)平移 T(0,-b)
2) θ=arctg(a)，顺时针旋转：R(-θ)
3)关于X轴的反射 $F_{X轴}$
4)旋转回去R(θ)
5)平移回去T(0,b)



Outline

- **Representation（对象在物理空间的表示）**
  - Vector space, Coordinate System, Change of Coordinate
  - Affine space, Frames System, Change of Frame
  - Homogeneous Coordinate
- **Affine Transformations*（仿射变换）**
  - Five Standard Transformation（标准变换）
  - Concatenation Transformation*（串联变换）
- **Applying Concatenation Transformation（应用串联变换）**
  - Eg1:Non-standard transformation非标变换
  - Eg2:Cumulative transformation累积/渐增变换

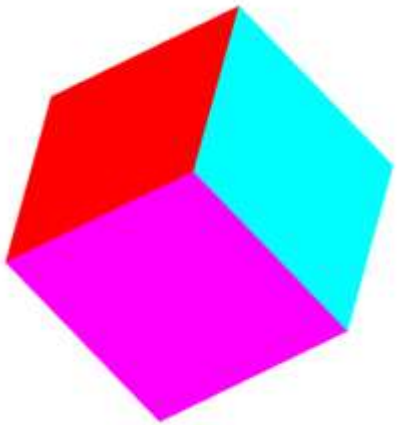
Cumulative Transformation

累积变换的例子: Virtual trackball 虚拟跟踪球模拟

- Smooth Rotation：From a practical standpoint, we are often want to use transformations to move and reorient an object smoothly

ref：Angel8ECode/ 04/trackball.*

- 让物体绕过中心的任意轴进行旋转;
- 旋转的任意轴是可以改变的，且由鼠标滑动方向来进行交互控制改变





The University of New Mexico

Cumulative Transformation（cont.）

累积变换的例子: Virtual trackball 虚拟跟踪球模拟（cont.）

- 第n帧画面中的顶点计算公式为： $P_n = R_n P_0$
  - 在CPU中算好当前帧的累积变换Rn，再发送给顶点着色器进行顶点变换r

```
<script id="vertex-shader" type="x-shader/x-vertex">
in  vec4 vPosition;
in  vec4 vColor;
Out  fColor;
uniform mat4 r;
void main(){
  vec4 p;
  gl_Position = r *vPosition;
  fColor = vColor;
}
```

- R_n是累积变换矩阵，多帧画面对应有各自的串联变换矩阵，形成变换序列 $R_0, R_1, \ldots, R_n$，且存在累积效应：$R_n = R_{cur} R_{n-1}$
  - R_{cur}是当前最近一次的任意轴旋转矩阵，有以下两种计算方式：
    - **find the Euler angles（欧拉角），use $R_{cur} = R_{iz} R_{iy} R_{ix}$**
      - **Not very efficient**
    - **find the rotation axis and angle ,use $R_{cur} = R(axis_i, angle_i)$,**
      - **Efficient**



Cumulative Transformation（cont.）

累积变换的例子: Virtual trackball 虚拟跟踪球模拟（cont.）

//关键代码-计算累积变换 $R_n = R_{cur}\ R_{n-1}$

```
Function render(){
  gl.clear( gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
    //鼠标键没有放开并继续移动时，则继续计算新的旋转轴和角度，得到新的一次旋转变换
    矩阵rotate(angle, axis)，累乘后传递rotationMatrix给着色器
   if(trackballMove) {  // 这里的rotationMatrix 就是Rn  (Rn = Rcur Rn-1)
      axis = normalize(axis); //下页讲如何计算axis
      //rotationMatrix = mult(rotationMatrix, rotate(angle, axis)); //error
      rotationMatrix = mult( rotate(angle, axis),rotationMatrix); //right
      gl.uniformMatrix4fv(rotationMatrixLoc, false, flatten(rotationMatrix));
    }
  gl.drawArrays( gl.TRIANGLES, 0, NumVertices );//绘制
  requestAnimFrame( render );//动画切换帧
}
```



The University of New Mexico

Cumulative Transformation（cont.）

Ex: Virtual trackball 虚拟跟踪球模拟（cont.）

- 当前旋转变换矩阵的实现代码：$R_{cur} = R(axis_i, angle_i)$

- ref: Angel8ECode/common\MV.js

```
/**生成绕任意轴旋转的变换矩阵*/
function rotate( angle, axis )
{
    if ( !Array.isArray(axis) ) {
        axis = [ arguments[1], arguments[2], arguments[3] ];
    }

    var v = normalize( axis );
    var x = v[0];
    var y = v[1];
    var z = v[2];

    var c = Math.cos( radians(angle) );
    var omc = 1.0 - c;
    var s = Math.sin( radians(angle) );

    var result = mat4(
        vec4( x*x*omc + c,   x*y*omc - z*s, x*z*omc + y*s, 0.0 ),
        vec4( x*y*omc + z*s, y*y*omc + c,   y*z*omc - x*s, 0.0 ),
        vec4( x*z*omc - y*s, y*z*omc + x*s, z*z*omc + c,   0.0 ),
        vec4()
    );

    return result;
}
```

- *思考：参数angle, axis如何计算？*

Cumulative Transformation（cont.）

The University of New Mexico

Ex: Virtual trackball 虚拟跟踪球模拟（cont.）

➢ $R_{cur} = R(axis_i, angle_i)$中参数axis 和 angle的计算方法

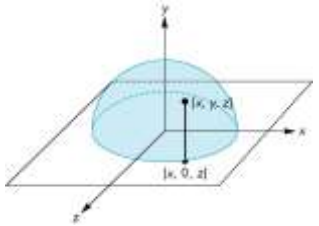
- 将轨迹球的位置和在正常鼠标垫上的位置联系起来，将鼠标位置p(x,z)反投影为半球坐标p(x,y,z)

$$Y=\sqrt{r^2 - x^2 - z^2},$$

if $r > |x| > 0, r > |z| > 0$

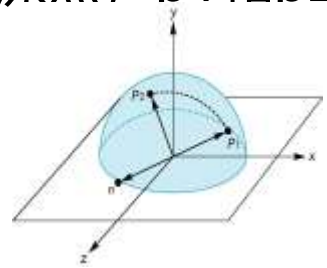


origin at center of ball



- 从鼠标上获得的两个点，将它们投射到上半球的点p1和点p2上，这些点决定了球体上的一个大圆
  - 从p1旋转到p2旋转轴axis，由原点，p1和p2决定的平面的法线表示

$$\mathbf{n} = \mathbf{p}_1 \times \mathbf{p}_2$$



  - p1,p2之间的角度angle，

当鼠标移动缓慢或频繁采样移动点时，

可以近似计算角度：$\theta \approx \sin\theta$

$$|\sin\theta| = \frac{|\mathbf{n}|}{|\mathbf{p}_1||\mathbf{p}_2|}$$



Cumulative Transformation（cont.）

Ex: Virtual trackball 虚拟跟踪球模拟（cont.）

- $R_{cur} = R(axis_i, angle_i)$中参数axis 和 angle的实现代码

```
/*根据输入的屏幕鼠标点击位置X,Y，将屏幕坐标2D转换为板球面上的3D坐标输出，并且单位化使得X2+Y2+Z2=1被mouseMotion调用*/
function trackballView( x,  y ) {
    var d, a;
    var v = [];
    v[0] = x;
    v[1] = y;
    d = v[0]*v[0] + v[1]*v[1];
    if (d < 1.0)
      v[2] = Math.sqrt(1.0 - d);
    else {
      v[2] = 0.0;
      a = 1.0 /  Math.sqrt(d);
      v[0] *= a;
      v[1] *= a;
    }
    return v;
}
```

```
/*mouseMotion当鼠标按下了并移动时，根据新的位置X,Y,转换为半球上三维坐标后，结合保留的上一次的三维位置，计算得出旋转轴及旋转角度，并且保留本次3D 位置。*/
function mouseMotion( x,  y)
{   var dx, dy, dz;
    var curPos = trackballView(x, y);

    if(trackingMouse) {
      dx = curPos[0] - lastPos[0];
      dy = curPos[1] - lastPos[1];
      dz = curPos[2] - lastPos[2];

      if (dx || dy || dz) {
         angle = -0.1 * Math.sqrt(dx*dx + dy*dy + dz*dz);
         axis[0] = lastPos[1]*curPos[2] - lastPos[2]*curPos[1];
         axis[1] = lastPos[2]*curPos[0] - lastPos[0]*curPos[2];
         axis[2] = lastPos[0]*curPos[1] - lastPos[1]*curPos[0];
         lastPos[0] = curPos[0];
         lastPos[1] = curPos[1];
         lastPos[2] = curPos[2];
      }
    }
    render();
}
```



Outline

- **Representation（对象在物理空间的表示）**
  - Vector space, Coordinate System, Change of Coordinate
  - Affine space, Frames System, Change of Frame
  - Homogeneous Coordinate
- **Affine Transformations*（仿射变换）**
  - Five Standard Transformation（标准变换）
  - Concatenation Transformation*（串联变换）
- **Applying Concatenation Transformation（应用串联变换）**
  - Eg1:Non-standard transformation非标变换
  - Eg2:Cumulative transformation累积变换
  - **...**

Summary

- **Homogeneous Coordinates Representation（齐次坐标）**
  - Point(x,y,z,h)，顶点表示，h!=0
  - Vector(x,y,z,0) 向量表示：h=0
- **Affine Transformation（仿射变换）**
  - Five Standard Transformation：Translation, Rotation, Scaling, Reflection, Shear.
  - Use Homogeneous Coordinates, Transformation is a 4*4 Matrix, 12 freedom
  - Use column vector representation , affine transformation then like:

$$M_{4*4} = \begin{bmatrix} sx & b & c & tx \\ d & sy & f & ty \\ g & h & sz & tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Concatenation Transformation（串联变换）**

Pn' = Mn*…*M2*M1* P0; //Mi is Standard Transformation

M=Mn*…*M2*M1; //M is Concatenation Transformation

- **Applying Concatenation Transformations（串联变换应用举例）**

Eg1:Non-standard transformation非标变换 M= N^{-1} $M_{standard}$ N **(N是变标准的过程矩阵)**

Eg2:Cumulative transformation累积变换 $M_n = M_{cur}\ M_{n-1}$

注：后续课讲MVP变换，即观察流水线中的各种坐标变换，其实都是串联变换的应用）

The University of New Mexico

Summary（cont.）

- 仿射变换是线性变换与平移变换的复合，在几何中用来描述“保持直线和平行性”的变换。
  - **具有保线性（直线，平行）**
  - **不具有保形性（角度，长度/距离）**
    - **只有“刚性变换”可以保障物体不变形：如 平移，旋转，反射 。**

变换	保持直线	保持平行	保持角度	保持长度
平移	✓	✓	✓	✓
旋转	✓	✓	✓	✓
反射	✓	✓	✓	✓
均匀缩放	✓	✓	✓	✗（比例变化）
非均匀缩放	✓	✓	✗	✗
错切	✓	✓	✗	✗



Summary（cont.）

- **扩展学习：**
  - 1：齐次坐标
    - B站视频“计算机图形学快速理解：齐次坐标 - Miolith”
  - 2：万向节锁（Gimbal Lock）
    - **万向节锁**指的是：当两个旋转轴对齐时，系统会**丢失一个旋转自由度**，导致无法再绕某个方向独立旋转。**用三个欧拉角描述三维旋转时，在某些姿态下丢失一个旋转自由度的数学现象，通常用四元数来避免它。**
    - B站视频：“无伤理解欧拉角中的“万向死锁”现象 。
  - 3：四元数（Quaternion）
    - 四元数是图形学中表示三维旋转最常用的数学工具。它扩展了复数，用 4 个数 就能干净地表示任意三维旋转，而且没有万向节锁。